AI-First Activity 1 — SRS with Prompt Engineering (FoodGO)

Course: CS 488 AI-First Software Engineering, Fall 2026 · **Team:** Hamza Ben Labiedh, Ali Chamakh, Mohamed Krid · **Date run:** 7 Nov 2026

Project idea used in every prompt: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

How the prompts were run. Each prompt was sent to Claude (Sonnet) in a **fresh session with no project context**, one session per prompt, so Step A and Step B never saw each other. Prompt chaining (Part 4B) used two prompts in one session. Part 5 pasted the same draft (our Part 2B + 3B + 4B outputs) into both prompts. Outputs below are the **AI's raw answers, unedited** except that their headings were demoted to fit this document. The comparisons are ours.

> **Honest note.** We ran this activity late (7 Nov), on the final version of the idea, after our SRS (Lab 2) was already written. So it did not shape the SRS; instead we compare its outputs with the SRS in the conclusion.

Summary of results

Part	Technique	Better version	Main reason
1 Stakeholders	Providing concept	**B** (slightly)	Covered the categories the definition named — maintainers and especially regulators — more completely; A was already broad
2 Functional requirements	One-shot	**B**	Identical format for all 10, each with priority and rationale; closer to one need per requirement
3 Non-functional requirements	Few-shot	**B** (clearly)	All 10 have a measurable target; A had several unmeasurable ones
4 User stories	Prompt chaining	**B** for clarity, **A** for breadth	B: traced to REQ IDs, exactly 3 Given/When/Then each; A: wider but inconsistent and invented scope
5 Quality review	Self-critique	**B** for testability, **A** for consistency	B listed, rewrote and justified; A caught more contradictions between requirements

---

Part 1 — Stakeholder Identification

Technique: Providing Concept (Explain-then-Ask) · **SRS section:** Stakeholders

---

Step A — Normal prompt

> List the stakeholders for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output A:

Stakeholders for FoodGO

1. Primary users (directly use the system)

- **Employees / customers** in Lac 3 / Zone Industrielle Le Kram: browse menus, pre-order before the morning deadline, pay (cash on delivery or Tunisian e-payment), and receive lunch at noon.
- **Restaurant owners / managers:** receive and prepare the pre-orders, manage menus and prices, and respect the cut-off.
- **Restaurant kitchen staff:** prepare the meals in batches according to the order list.
- **Restaurant delivery staff (drivers/couriers):** deliver at noon with the restaurant's own staff and collect cash when needed.
- **FoodGO platform administrators:** manage restaurants, workplaces, zones, menus, and monitor orders.

2. Client / business side

- **Project owner / sponsor:** funds the project and decides the business rules, such as the order deadline and new order states.
- **Product owner / business analyst:** defines requirements and priorities.
- **Platform operator (the company running FoodGO):** earns revenue, handles commissions or fees, and runs support.

3. Technical team

- **Developers (frontend, backend, database):** build and maintain the app.
- **UI/UX designer:** designs the ordering experience.
- **QA / testers:** verify the deadline logic, payment flows, and restaurant scoping.
- **DevOps / hosting provider:** deployment, CI, uptime, and security.
- **Customer support team:** handles complaints, missing or wrong orders, and refunds.

4. Workplace and local community

- **Employers / HR / office managers** of companies in the zone: may approve the service, set delivery points, and benefit from reduced lunch absences.
- **Building or site managers / security guards:** allow delivery access to the premises.
- **Zone/area authority and local municipality (Tunis, Lac 3 / Le Kram):** interested in reduced lunchtime traffic.
- **Residents, commuters, and other road users:** indirect beneficiaries of less congestion.

5. External services and regulators

- **E-payment providers / banks** (e.g., Tunisian payment gateways such as Monetique Tunisie, Konnect, Flouci, or bank cards): process online payments.
- **SMS provider:** sends OTP and login codes.
- **Regulators and authorities:** the Tunisian data protection authority (INPDP) for personal data, the tax authority (invoices and VAT), the health and hygiene inspection authority (food safety), and the consumer protection authority.
- **Competitors / alternatives:** other delivery apps, canteens, and street food vendors (they affect positioning, though they are not formal stakeholders).

6. Academic context (if this is a course project)

- **Instructor / supervisor and evaluators.**
- **Project team members.**

Key interests and potential conflicts

- Employees want speed, choice, and low cost, but they are bound by the morning deadline.
- Restaurants want predictable volume and fair fees, but they must use their own delivery staff.
- Employers want fewer long lunch breaks and are concerned about disruption.
- Authorities want less traffic and compliance with food, tax, and data rules.
- Operators want growth and reliable payments, while limiting cash-handling risk.

Step B — Providing Concept (Explain-then-Ask)

> In software engineering, stakeholders are people or groups affected by the project, including end-users, customers, maintainers, regulators, and others. Based on this definition, list all potential stakeholders for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output B:

Stakeholders for FoodGO

1. Primary users (use the system directly)

- **Employees / customers** in Lac 3 / Zone Industrielle Le Kram who place pre-orders, pay and receive lunch.
- **Restaurant owners / managers** who accept orders, set menus and prices, and meet the morning deadline.
- **Restaurant kitchen staff** who prepare the orders.
- **Restaurant delivery staff** who deliver at noon with the restaurant's own staff.
- **FoodGO administrators / operators** who manage restaurants, workplaces, menus and the zone, and handle support.

2. Customer and business-side stakeholders

- **Employers / companies in the zone** (HR, office managers, receptionists or security at the entrances). They host deliveries, may encourage or subsidise the service, and care about staff productivity and lunch breaks.
- **FoodGO owners / founders / sponsors / investors** who fund the project and set its business goals.
- **Product owner / business analysts** who define requirements and rules.
- **Competitors**, such as other delivery apps and the canteens or cafeterias near workplaces (indirectly affected).

3. Technical and delivery-side stakeholders

- **Development team** (developers, testers, QA, UI/UX designers).
- **DevOps / hosting and infrastructure providers**, for deployment, monitoring and backups.
- **Maintainers and support staff**, who fix bugs, add features and answer users.
- **Project manager / teacher / course supervisor**, if it is an academic project.
- **Future contributors**, who rely on the documentation.

4. External service providers

- **Tunisian e-payment gateways and banks**, such as Monetique Tunisie, Flouci, Paymee or card networks.
- **SMS provider** for OTP authentication.
- **AI / forecasting service provider**, if used.
- **Mapping and cloud services**, if used.

5. Regulators and legal or public bodies

- **Data protection authority (INPDP, Tunisia)**, for personal data such as phone numbers, names and workplaces.
- **Central Bank of Tunisia / financial regulators**, for e-payment compliance.
- **Tax authorities**, for invoicing, VAT and cash-on-delivery revenue.
- **Health and food safety authorities** (Ministry of Health / hygiene inspections), covering restaurants and food handling.
- **Municipality / local authorities and traffic or transport authorities**, for traffic rules, delivery vehicles and parking.
- **Labour regulators**, for restaurant delivery staff working conditions.
- **Consumer protection bodies**, for orders, refunds and disputes.

6. Indirectly affected parties

- **Other road users and residents of the zone**, who benefit from less lunchtime traffic.
- **Zone / industrial area management** (the area's management authority), because its goal is traffic reduction.
- **Neighbouring restaurants and shops** that are not on the platform and may lose business.
- **Suppliers of the restaurants** (food and packaging), since order volume changes.
- **Environment and community**, through lower emissions and less congestion.
- **Employees' families and colleagues** who don't order but are affected by the lunch routine.
- **Employees with dietary, religious or accessibility needs** (halal, allergies, disabilities), who need to be considered in design.

Key conflicts to manage: employees want late ordering and flexibility, while restaurants need an early deadline. Restaurants want to maximise orders, while the zone wants less traffic. Cash on delivery is convenient but risky for restaurant staff, who carry cash and bear the risk of refusals.

Comparison

Criterion	A — normal	B — providing concept
Breadth	~25 stakeholders in 6 groups	~40 stakeholders in 6 groups
Organisation	Users, business, technical team, community, external/regulators, academic	Follows the definition's own categories: users, customers/business, maintainers, external providers, **regulators** (its own group), indirectly affected
Relevance	High; already mentions traffic, INPDP, tax and food-safety bodies	High; adds the Central Bank (e-payment), labour and traffic authorities, neighbouring shops, suppliers, dietary needs
Conflicts	5 interest conflicts listed	3 sharper conflicts (deadline vs. flexibility, orders vs. traffic, cash risk)

Verdict: B is broader and more systematically organised, because the definition told the model which categories to cover (maintainers, regulators), and it filled each one. The gap was smaller than we expected: a modern model already gives a broad list to the plain prompt. The concept prompt mainly guarantees **coverage** rather than adding a lot of new insight. A few of B's additions (employees' families) are a stretch.

Part 2 — Functional Requirements

Technique: One-Shot Prompting · **SRS section:** Functional Requirements

---

Step A — Normal prompt

> Generate 10 functional requirements for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output A:

FoodGO: 10 Functional Requirements

FR-01: User registration and authentication. The system shall let employees register and log in with a phone number verified by an SMS one-time code (OTP). Each account shall be linked to a workplace address in the Lac 3 / Zone Industrielle Le Kram zone.

FR-02: Delivery zone validation. The system shall accept only workplaces and restaurants located in the Lac 3 / ZI Le Kram zone. Users with an address outside this zone shall be refused at registration or checkout, with a clear message.

FR-03: Restaurant and menu browsing. The system shall display the participating restaurants and their menus (dishes, descriptions, prices, photos, availability). Users shall be able to search and filter by restaurant, category or price.

FR-04: Cart and order placement. The system shall let an employee add dishes from a single restaurant to a cart, adjust quantities, add notes, and place a lunch order for the same day, with a total including any fees.

FR-05: Pre-order deadline enforcement. The system shall accept orders only before each restaurant's morning cut-off time (for example 10:30 for Kram Snack and 23:30 for Dar Lella, as configured). After the deadline, ordering shall be disabled and users shall see the remaining time or a "closed" status.

FR-06: Payment method selection. At checkout the user shall choose either cash on delivery or Tunisian e-payment (for example card payment via a local gateway such as Monétique Tunisie or Edinar). The system shall record the payment status and shall not store card data.

FR-07: Order confirmation and tracking. The system shall confirm each order with a reference number and show its status (for example placed, preparing, out for delivery, delivered). Users shall be able to view their current and past orders. New states beyond the agreed set require a stakeholder decision.

FR-08: Restaurant order management. Restaurant staff shall have a dashboard showing the day's orders after the deadline, grouped by delivery location and with totals per dish, so they can prepare and deliver with their own staff at noon. Restaurants shall be able to manage their menu, prices, availability and opening hours.

FR-09: Delivery grouping and noon delivery schedule. The system shall group orders by workplace or building and assign them to the restaurant's noon delivery window. Delivery staff shall be able to mark orders as delivered and, for cash orders, confirm cash collected.

FR-10: Notifications. The system shall send notifications (SMS or in-app) to employees for order confirmation, deadline reminders and delivery status, and to restaurants for new orders and the end-of-day summary.

Optional extras: an admin back office for managing restaurants, workplaces and users; and an advisory demand forecast and traffic-reduction report that uses no customer personal data.

Step B — One-Shot Prompting

> Here is the style I want: Example: REQ-001: The system must allow users to register via email and password. Priority: High. Rationale: Required for authentication. Now generate 10 functional requirements in this format for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning

> deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output B:

REQ-001: The system must allow employees to register and log in using their mobile phone number verified by an SMS one-time code. Priority: High. Rationale: Required for authentication and to link every order to an identifiable employee.

REQ-002: The system must restrict the service to the Lac 3 / Zone Industrielle Le Kram area by requiring each restaurant and each employee's workplace delivery address to belong to that zone. Priority: High. Rationale: FoodGO serves a single area, and delivery by restaurant staff is only feasible inside it.

REQ-003: The system must display the list of participating restaurants with their menus, item prices, and daily availability. Priority: High. Rationale: Employees need this information to choose and compose a lunch order.

REQ-004: The system must allow an employee to build a cart with items from a single restaurant and place a pre-order for that day's noon delivery. Priority: High. Rationale: This is the core purpose of the app and keeps each delivery tied to one restaurant's staff.

REQ-005: The system must enforce each restaurant's morning ordering deadline (for example, 10:30 for Kram Snack and 23:30 for Dar Lella as configured) and reject or hide orders placed after it. Priority: High. Rationale: Restaurants need a fixed cut-off to prepare the exact quantities and plan their noon deliveries.

REQ-006: The system must let the employee choose a payment method at checkout, either cash on delivery or Tunisian e-payment. Priority: High. Rationale: These are the two payment options defined for the service.

REQ-007: The system must process e-payment through a Tunisian payment gateway and confirm the order only after the payment is successfully authorized. Priority: High. Rationale: Prevents unpaid orders and ensures payment status is reliable.

REQ-008: The system must show each employee an order confirmation and let them view the status and history of their orders (for example, placed, preparing, out for delivery, delivered). Priority: Medium. Rationale: Gives transparency and reduces calls to restaurants about order progress.

REQ-009: The system must provide each restaurant with a dashboard listing the day's confirmed orders grouped by delivery address or workplace, with item totals and the payment method (cash or paid online). Priority: High. Rationale: Restaurants prepare food and organize their own staff delivery at noon, and must know how much cash to collect.

REQ-010: The system must allow restaurant staff to update an order's status, including marking it as delivered and, for cash orders, as paid on collection. Priority: Medium. Rationale: Keeps order tracking accurate and supports daily reconciliation of cash and online payments.

Comparison

Criterion	A — normal	B — one-shot
Structure	Titled "FR-01: Name." paragraphs; no priority or rationale; extra "optional" items	Exactly the example's format for all 10: `REQ-xxx: The system must …` `Priority: … Rationale: …`
Clarity	Several requirements bundle features (FR-03 browsing + search + filter; FR-08 dashboard + menu management)	One need per requirement in most cases (REQ-006 choice and REQ-007 processing are separate)
Consistency	"shall" style, varying length and detail	Uniform "must" style; each rationale explains **why**, which helps prioritising
Traceability	IDs, but nothing to rank them	IDs + priority + rationale, ready for a backlog

Verdict: B is clearly more structured and consistent. One example was enough to fix the format, add priority and rationale, and make requirements more atomic. Neither output is perfect: both copied an inconsistent example (a "morning" deadline of 23:30 for Dar Lella), which Part 5 later caught.

Part 3 — Non-Functional Requirements

Technique: Few-Shot Prompting · **SRS section:** Non-Functional Requirements

---

Step A — Normal prompt

> List 10 non-functional requirements for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output A:

10 Non-Functional Requirements for FoodGO

1. **Performance (peak load):** Around the morning deadline, the system must handle a burst of concurrent orders (e.g. 500 users placing orders within a 30-minute window). Menu pages and order confirmation must load in under 2 seconds for 95% of requests on a 4G connection.
2. **Deadline accuracy and timing:** Each restaurant's order cutoff must be enforced server-side, using a single synchronized clock in Tunisia time (CET). No order may be accepted or modified after the cutoff, and the time shown to users must match the server's.
3. **Availability:** The service must be available at least 99.5% during the ordering window (before the morning deadline) and during delivery hours (around noon). Planned maintenance must be scheduled outside those hours.

4. **Reliability and data integrity:** Accepted orders must never be lost or duplicated, even if the server fails or the user's connection drops. Order submission must be idempotent and use transactions. Daily backups must be kept, with recovery within 4 hours.

5. **Security:** All traffic must use HTTPS/TLS. Passwords must be hashed. Sessions must be protected with expiry and rate limiting. The system must be protected against OWASP Top 10 risks (injection, XSS, CSRF). Role-based access must ensure restaurants only see their own orders.

6. **Payment security and compliance:** Online payments must go through a certified Tunisian e-payment gateway. The app must never store card data (PCI DSS scope stays with the gateway). Payment callbacks must be verified and logged. Cash-on-delivery orders must be recorded reliably for reconciliation.

7. **Privacy and data protection:** Personal data (name, phone, workplace, order history) must be collected only as needed and handled in line with Tunisian Law No. 2004-63 on personal data protection. Users must be able to view and delete their data. Data shared with restaurants must be limited to what is needed for delivery.

8. **Usability and accessibility:** The interface must be simple enough for a new user to place an order in under 2 minutes without training. It must follow basic accessibility guidelines (WCAG 2.1 AA: contrast, labels, keyboard use). It must be available in French and Arabic, with English optional.

9. **Compatibility and mobile-first design:** The app must work well on low-to-mid-range smartphones and current versions of Chrome, Safari, Firefox and Edge. It must be responsive from 360px screen width upward and remain usable on slow or unstable mobile networks.

10. **Maintainability, scalability and observability:** The code must be modular and documented, with automated tests (e.g. at least 60% coverage) and a CI pipeline. Adding a new restaurant or area must need only configuration, not code changes. Structured logs, error monitoring and alerts must allow quick diagnosis of problems, especially near the deadline.

Step B — Few-Shot Prompting

> Here are examples of good non-functional requirements: Performance: The system must respond to user clicks within 2 seconds. Usability: The app must achieve a System Usability Scale score of at least 75. Reliability: The system must have 99.9% uptime over a 30-day period. Based on these examples, write 10 non-functional requirements for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output B:

1. Performance: The menu and restaurant list pages must load in under 3 seconds on a 4G mobile connection for 95% of requests.

2. Performance: The order placement action (submit to confirmation screen) must complete within 2 seconds for 95% of orders, including during the peak 30 minutes before the morning ordering deadline.
3. Scalability: The system must support at least 500 concurrent users and 1,000 orders submitted within the final 15 minutes before the deadline without errors or a response-time increase of more than 50%.
4. Reliability: The system must achieve 99.5% uptime over a 30-day period, and 99.9% during the ordering window (from the opening of orders until the deadline).
5. Deadline integrity: The system must enforce each restaurant's ordering deadline to within 1 second of the server clock. No order may be accepted after the deadline, and no order accepted before it may be lost.
6. Security: All traffic must use HTTPS (TLS 1.2 or higher). Passwords and one-time SMS codes must be stored and transmitted only in hashed or expiring form, with codes expiring within 5 minutes.
7. Payment security and compliance: The system must never store card numbers or CVV codes. Tunisian e-payment must be processed through a certified payment provider, and 100% of payment results (success or failure) must be recorded against the order. Cash-on-delivery orders must be clearly flagged as unpaid until delivery is confirmed.
8. Privacy and data protection: Personal data (name, phone number, workplace, delivery address) must be collected only as needed for delivery, be visible only to the customer and the restaurant that fulfils the order, and be handled in line with Tunisian data protection law (Law No. 2004-63). Any AI-generated demand forecast must be built without customer-identifying data.
9. Usability: The app must achieve a System Usability Scale score of at least 75, and 90% of first-time users must be able to place an order in under 3 minutes without help.
10. Compatibility and accessibility: The app must work on the latest two versions of Chrome, Safari, Firefox and Edge, and on screens from 360 px wide. It must meet WCAG 2.1 level AA for contrast and keyboard/screen-reader access, and must be usable in French and Arabic.

Comparison

Criterion	A — normal	B — few-shot
Specificity	Good in places (2 s, 99.5 %, 4 h recovery)	Every requirement names a concrete target
Measurability	Several cannot be tested: "simple enough … without training", "modular and documented", "follow basic accessibility guidelines"	10/10 measurable: 3 s / 2 s at the 95th percentile, 500 users and 1,000 orders in 15 min, 99.5 % / 99.9 %, deadline within 1 s, TLS 1.2+, codes expire in 5 min, SUS ≥ 75, 90 % of first-timers under 3 min, WCAG 2.1 AA
Style	Mixed: some are lists of practices (OWASP, CI) rather than requirements	Same "category: must … number" pattern as the examples

Verdict: B is much more specific and measurable. The three examples taught the model that a good NFR has a number and a measurement window. This was the biggest improvement of the five

techniques. Caution: B's numbers (500 users, SUS 75) are **plausible assumptions, not facts about FoodGO**, so they need confirming with stakeholders.

Part 4 — User Stories & Acceptance Criteria

Technique: Prompt Chaining · **SRS section:** User Stories

Step A — Normal prompt

> Create user stories with acceptance criteria for this project: FoodGO — a web app where employees in the Lac 3 / Zone Industrielle Le Kram area of Tunis pre-order lunch from local restaurants before a morning deadline; each restaurant delivers with its own staff at noon, to reduce lunchtime traffic in the area. Payment is cash on delivery or Tunisian e-payment.

Output A:

FoodGO: User Stories with Acceptance Criteria

Roles: Employee (customer), Restaurant staff, Restaurant manager, Workplace/company admin, Platform admin. **Key rules assumed:** one service area (Lac 3 / ZI Le Kram), order before a morning deadline (set per restaurant), delivery at noon by the restaurant's own staff, payment by cash on delivery (COD) or Tunisian e-payment.

Epic 1: Account and Access

US-1.1 Register and log in As an employee, I want to sign up and log in with my phone number so that I can order quickly.

- Given I enter a valid Tunisian mobile number, when I request a code, then an SMS code is sent and I can enter it to log in.
- Given I enter a wrong or expired code, then I see a clear error and can request a new code.
- Given I am logged in, my session stays active until I log out or it expires.

US-1.2 Set my delivery location As an employee, I want to choose my workplace so that my order is delivered to the right address.

- Given I register, I must select a workplace in the Lac 3 / ZI Le Kram zone.
- Given I enter an address outside the zone, then I am told FoodGO does not serve that area and cannot continue.
- I can add delivery notes (building, floor, reception name).

Epic 2: Browsing and Ordering

US-2.1 See open restaurants As an employee, I want to see restaurants that deliver to my workplace so that I can pick lunch.

- Only restaurants serving my zone are listed.
- Each card shows name, cuisine, and the order deadline for today.
- Restaurants whose deadline has passed are shown as "Closed for today" and cannot be ordered from.

US-2.2 View a menu As an employee, I want to see a restaurant's menu with prices so that I can decide what to order.

- Items show name, description, price in TND, and availability.
- Unavailable items are greyed out and cannot be added.

US-2.3 Build a cart As an employee, I want to add, remove and change quantities so that I can adjust my order.

- The cart total updates immediately.
- A cart contains items from one restaurant only; adding from another restaurant prompts me to start a new cart.
- Quantity cannot be below 1 or above a defined maximum.

US-2.4 Place an order before the deadline As an employee, I want to confirm my order before the morning deadline so that it is delivered at noon.

- Given the time is before the restaurant's deadline, when I confirm, then the order is created and I receive a confirmation with order number and delivery time (noon).
- Given the deadline has passed, when I try to confirm, then the order is rejected with a message explaining the cutoff.
- The deadline is checked on the server, not only in the browser.
- Placing the same order twice by double-tap does not create duplicates.

US-2.5 See the deadline countdown As an employee, I want to see how much time is left to order so that I do not miss the cutoff.

- A countdown is visible on restaurant pages and in the cart.
- When time runs out, the page switches to the closed state without requiring a refresh.

US-2.6 Modify or cancel my order As an employee, I want to change my order before the deadline so that I can fix mistakes.

- Given the deadline has not passed, I can edit items or cancel and see the updated total.
- After the deadline, edit and cancel actions are disabled.
- (Note: new order states beyond the agreed lifecycle need a stakeholder decision.)

Epic 3: Payment

US-3.1 Pay cash on delivery As an employee, I want to choose cash on delivery so that I can pay when my food arrives.

- COD is selectable at checkout and the order summary shows "Pay X TND on delivery".
- The restaurant sees that the order is unpaid/COD.
- Staff can mark the cash as collected at delivery.

US-3.2 Pay online with Tunisian e-payment As an employee, I want to pay by a Tunisian e-payment method so that I do not need cash.

- Given I choose e-payment, I am redirected to the payment provider and returned to FoodGO with the result.
- On success, the order is marked paid and I get a receipt.
- On failure or abandonment, the order is not confirmed as paid and I can retry or switch to COD before the deadline.
- No card data is stored by FoodGO.

US-3.3 Receipt and history As an employee, I want to see past orders and receipts so that I can track my spending.

- The history lists date, restaurant, items, total, payment method and status.

Epic 4: Delivery and Tracking

US-4.1 Order status — As an employee, I want to see my order status so that I know when to expect lunch.

- Statuses shown: placed, being prepared, out for delivery, delivered. The expected delivery time (noon) is displayed.

US-4.2 Notifications — As an employee, I want a notification when my order is confirmed and when it is on its way so that I am ready to receive it.

- I receive an SMS or in-app message on confirmation and on dispatch.

US-4.3 Delivery by restaurant staff — As a restaurant staff member, I want a delivery list grouped by workplace so that I can deliver efficiently at noon.

- The list shows order number, customer name, workplace address, notes, items, and amount to collect (for COD). I can mark each order as delivered.

Epic 5: Restaurant Management

US-5.1 Manage menu — As a restaurant manager, I want to add, edit and hide menu items so that customers see accurate offers.

- I can set name, description, price, photo and availability. Changes appear for customers immediately; items hidden today do not affect already placed orders.

US-5.2 Set daily deadline — As a restaurant manager, I want to set my ordering deadline so that I have time to prepare.

- The deadline must be a time in the morning within an allowed range. Changes apply to future orders only and do not invalidate confirmed ones.

US-5.3 Daily preparation summary — As a restaurant manager, I want to see totals per item after the deadline so that I can prepare the right quantities.

- After the deadline, the summary shows total quantity per item and the number of orders. The list is final and no new orders can be added.

US-5.4 Set delivery capacity — As a restaurant manager, I want to limit the number of orders per day so that my staff can deliver by noon.

- When the limit is reached, the restaurant shows "Fully booked" and accepts no further orders.

Epic 6: Workplace Admin

US-6.1 Group ordering overview — As a workplace admin, I want to see which employees ordered today so that deliveries to my building are coordinated.

- I see counts per restaurant, not employees' payment details.

Epic 7: Platform Administration

US-7.1 Manage restaurants and workplaces — As a platform admin, I want to add and deactivate restaurants and workplaces so that the service stays accurate.

- A new restaurant or workplace must belong to the Lac 3 / ZI Le Kram zone; other areas are rejected. Deactivated restaurants disappear from customer listings; existing orders are unaffected.

US-7.2 Monitor orders and payments — As a platform admin, I want a dashboard of daily orders and payment status so that I can spot problems.

- The dashboard shows orders per restaurant, COD vs e-payment split, and failed payments.

US-7.3 Traffic impact reporting — As a platform admin, I want to see how many deliveries are consolidated per restaurant so that I can show the reduction in lunchtime traffic.

- The report shows orders per restaurant per day and average orders per delivery run.

Non-Functional Stories

- **Performance:** As an employee, I want pages to load in under 3 seconds on a mobile connection so that ordering is quick.
- **Security:** As an employee, I want my data protected so that my phone number and orders are private. Access is restricted by role; employees can see only their own orders; restaurants see only their own.
- **Reliability near the deadline:** As a restaurant, I want the system to handle many orders just before cutoff so that none are lost.
- **Localization:** As an employee, I want the interface in French and Arabic (and optionally English) with prices in TND.
- **Accessibility and mobile:** The app works on small phone screens and meets basic accessibility guidelines.
- **Privacy:** Any AI-based demand forecast is advisory only and uses no customer personal data.

Suggested Priority (MVP)

1. Must have: 1.1, 1.2, 2.1 to 2.4, 3.1, 3.2, 4.1, 4.3, 5.1, 5.2, 5.3, 7.1
2. Should have: 2.5, 2.6, 3.3, 4.2, 5.4, 7.2
3. Could have: 6.1, 7.3, forecasting

Step B — Prompt Chaining

> Prompt 1: "Generate 6 user stories for these functional requirements: [the Part 2 one-shot output, REQ-001 to REQ-010]." Prompt 2 (same conversation): "For each user story, write 3 acceptance criteria using Given/When/Then format."

Output B:

Chained prompt 1 — answer:

Six user stories covering REQ-001 to REQ-010 (related requirements are combined so that each story delivers one coherent piece of value).

US-1 — Sign up and log in with my phone (REQ-001) As an employee in the Lac 3 / Zone Industrielle Le Kram area, I want to register and log in with my mobile phone number verified by an SMS one-time code, so that I can access the app securely and every order I place is linked to me.

US-2 — Browse nearby restaurants and menus (REQ-002, REQ-003) As an employee, I want to see only restaurants located in my zone, with their menus, item prices and daily availability, and to give a workplace address that is in that zone, so that I can choose a lunch that can actually be delivered to me.

US-3 — Pre-order lunch before the cut-off (REQ-004, REQ-005) As an employee, I want to build a cart with items from a single restaurant and place a pre-order for today's noon delivery before that restaurant's morning deadline, so that my lunch is prepared and delivered with the exact quantities the restaurant planned.

US-4 — Pay by cash or online (REQ-006, REQ-007) As an employee, I want to choose at checkout between cash on delivery and Tunisian e-payment, and have my order confirmed only when an online payment is authorized, so that I can pay the way I prefer and know my order is valid.

US-5 — Follow my orders (REQ-008) As an employee, I want to receive an order confirmation and to view the status (placed, preparing, out for delivery, delivered) and the history of my orders, so that I know where my lunch is without calling the restaurant.

US-6 — Run the day's orders as a restaurant (REQ-009, REQ-010) As a restaurant staff member, I want a dashboard of the day's confirmed orders grouped by delivery address or workplace, with item totals and payment method, and the ability to update each order's status (including delivered and, for cash orders, paid on collection), so that I can prepare food, organize noon deliveries, collect the right cash, and reconcile payments.

Chained prompt 2 — answer:

US-1 — Sign up and log in with my phone

1. Given I am a new user on the registration screen, When I enter a valid mobile phone number and request a code, Then the system sends a one-time code by SMS to that number.
2. Given I have received a one-time code, When I enter the correct code before it expires, Then my account is created or I am logged in, and I am taken to the restaurant list.
3. Given I have requested a code, When I enter an incorrect or expired code, Then I am not logged in and I see a clear error message with the option to request a new code.

US-2 — Browse nearby restaurants and menus

1. Given I am logged in, When I open the restaurant list, Then I see only participating restaurants that belong to the Lac 3 / Zone Industrielle Le Kram zone, each with its menu, item prices and today's availability.
2. Given a menu item is unavailable today, When I view that restaurant's menu, Then the item is shown as unavailable (or hidden) and cannot be added to my cart.
3. Given I am entering my workplace delivery address, When the address does not belong to the Lac 3 / Zone Industrielle Le Kram zone, Then the system rejects it and explains that the service only covers that zone.

US-3 — Pre-order lunch before the cut-off

1. Given my cart contains items from one restaurant, When I confirm the order before that restaurant's morning deadline (for example 10:30 for Kram Snack), Then the pre-order is placed for today's noon delivery.
2. Given my cart already contains items from one restaurant, When I try to add an item from a different restaurant, Then the system prevents it and tells me a cart can hold items from a single restaurant only.
3. Given the restaurant's ordering deadline has passed, When I try to place an order with that restaurant, Then the order is rejected (or the restaurant is hidden) and I see a message that ordering is closed for today.

US-4 — Pay by cash or online

1. Given I am at checkout with a valid cart, When I view the payment options, Then I can choose either cash on delivery or Tunisian e-payment.
2. Given I selected e-payment, When the Tunisian payment gateway successfully authorizes my payment, Then my order is confirmed and marked as paid online.
3. Given I selected e-payment, When the gateway declines or fails to authorize the payment, Then my order is not confirmed and I am told the payment failed, with the option to retry or switch to cash on delivery.

US-5 — Follow my orders

1. Given my order has been confirmed, When checkout completes, Then I see an order confirmation showing the restaurant, items, total, payment method and delivery address.
2. Given I have an active order, When the restaurant changes its status (for example to preparing or out for delivery), Then I can see the updated status when I open the order.
3. Given I have placed orders on previous days, When I open my order history, Then I see my past orders with their date, restaurant, total and final status.

US-6 — Run the day's orders as a restaurant

1. Given I am logged in as staff of a restaurant, When I open the dashboard, Then I see only my restaurant's confirmed orders for today, grouped by delivery address or workplace, with item totals and each order's payment method (cash or paid online).
2. Given an order is out for delivery, When I mark it as delivered, Then its status updates to delivered and the employee can see this in their order status.
3. Given a cash-on-delivery order has been delivered, When I mark it as paid on collection, Then the order is recorded as paid so the day's cash and online payments can be reconciled.

Comparison

Criterion	A — single prompt	B — chained (stories, then criteria)
Size	~22 stories in 7 epics + non-functional stories + MVP priority	6 stories × 3 criteria
Format	Mixed: some criteria are Given/When/Then, many are plain bullets; 1–4 criteria per story	Exactly 3 Given/When/Then criteria per story, all testable
Traceability	No link to requirements	Each story names the REQ IDs it covers (REQ-001 … REQ-010)
Scope	**Invents features** we never required: order modification/cancellation, delivery capacity, workplace admin	Stays inside the agreed requirements

Verdict: chaining (B) is clearer and more consistent; the single prompt (A) is more complete in breadth but less reliable. Chaining worked because step 1 fixed *what* to cover (our own requirements) and step 2 fixed *how* to write the criteria. A's invented order cancellation is a real risk for us: our specification leaves cancellation **unresolved**, so a single-shot answer could have slipped an undecided business rule into the SRS.

Part 5 — Quality Review & Refinement

Technique: Self-Critique Prompting · **SRS section:** Quality-checked Requirements

---

Step A — Normal prompt

> Review my requirements and make them better. [pasted: the Part 2 one-shot FRs + Part 3 few-shot NFRs + Part 4 chained user stories]

Output A:

Review of your requirements

Overall this is a strong draft. The requirements are mostly testable, each has a priority and rationale, the NFRs carry real numbers, and the stories trace back to REQ IDs. The improvements below are about precision, gaps and consistency, in rough order of importance.

1. Problems to fix

REQ-005: the deadline example is inconsistent. "Morning ordering deadline ... 23:30 for Dar Lella" is a night time, not a morning one. Either the deadline is not always in the morning or one value is a typo. Also, "reject or hide" is ambiguous. Pick one behaviour (for example: the restaurant stays visible but marked "Closed for today" and checkout is rejected). Move the specific times into configuration data rather than the requirement text.

REQ-004 versus REQ-005 vs. "noon delivery". The delivery time is fixed at noon, but a restaurant with a 23:30 cut-off cannot serve "that day's noon delivery" if the cut-off is the night before. State explicitly which day's delivery a late-evening order belongs to.

REQ-006 and REQ-007 overlap. Payment choice and payment processing are split across two requirements, and "Tunisian e-payment" is undefined. Name the supported methods or say "a gateway to be selected" so it is testable. Also state what happens to an order when the payment is pending or times out.

REQ-008 and REQ-010 disagree on states. REQ-008 lists placed, preparing, out for delivery, delivered. REQ-010 only mentions delivered and paid on collection. Define the full allowed status list and the legal transitions (for example, placed to preparing to out for delivery to delivered). Say explicitly that cancelled, rejected and failed are out of scope for now, so nobody assumes them.

NFR 6: passwords are mentioned, but the system uses phone plus SMS code. There are no passwords in REQ-001. Remove "passwords" or add the requirement that introduces them. "Stored and transmitted only in hashed or expiring form" is also muddled: a code in transit cannot be hashed. Rewrite as: codes are single-use, expire in 5 minutes, are stored hashed, and are never logged.

2. Gaps (missing requirements)

- **Authentication abuse:** limits on code requests and on wrong attempts (for example, 5 codes per hour per number, lock after 5 wrong tries).
- **Restaurant accounts:** REQ-009 and US-6 assume restaurant staff log in, but no requirement says how they are created or authenticated, and that each can see only their own restaurant's data. This is the most important access-control rule in the system, so it deserves its own requirement.
- **Restaurant administration:** who maintains menus, prices and daily availability (REQ-003 displays them, but nothing lets anyone change them).
- **Employee workplace data:** where the workplace address is captured and edited (it appears only in REQ-002 and US-2).
- **Price integrity:** the price charged must be the price shown at order time, and totals must be calculated on the server.
- **Daily reset:** what happens to carts left over from the previous day.
- **Notifications:** REQ-008 hints at them ("confirmation") but not whether SMS, in-app or both.
- **Cash risk:** a cap on cash order value, or a rule for no-shows on cash orders, if the stakeholders want one. Mark this as a question, not a decision.

3. Wording improvements

- Replace "for example" inside requirements (REQ-005, REQ-008) with fixed values. Examples belong in notes; a requirement should be verifiable as written.
- Use "shall" consistently and one requirement per statement. REQ-002 contains two rules (restaurants in zone and addresses in zone); REQ-010 contains two (delivered and paid). Split them so each can be tested and traced separately.
- Add an ID scheme to the NFRs (NFR-001 and so on). Right now they are numbered 1 to 10 and cannot be cited cleanly in the stories or a traceability matrix.
- Add a "Source/Stakeholder" and "Status" column, or at least a short note on who approved each rule, since the project forbids inventing business rules.
- NFR 3 mixes two measures. Separate the concurrency target from the "response time increase of 50%" and give a baseline ("compared with the 500-user baseline").
- NFR 9: "SUS of 75" needs a sample size and test conditions (for example, at least 10 participants), otherwise it cannot be verified.
- NFR 10: "latest two versions" of four browsers is a heavy test matrix. Confirm it is intended, and say whether Arabic requires right-to-left layout.
- NFR 8: say "Law No. 2004-63" is subject to legal review, since compliance claims should be confirmed by someone qualified. Also add retention, for example how long order and phone data are kept.

4. User story fixes

- Stories 1 to 6 are well scoped, but US-2 mixes two ideas (browse restaurants and validate a workplace address). Consider moving address validation into US-1 (profile set-up) or its own story.
- The stories use "As an employee ... so that" correctly, but US-6's "so that" lists four goals. Trim to the main one, or split into "view orders" and "update status".
- Acceptance criteria are all Given/When/Then, which is good. Add negative and edge cases: code request rate limit, order submitted at exactly the deadline second, payment gateway timeout, double-tap on submit (duplicate order), and a restaurant staff member trying to open another restaurant's order.
- US-3 criterion 1 repeats "10:30 for Kram Snack"; keep the numbers in one place.
- Add a traceability table (REQ to US to NFR to test) at the end. It will expose any requirement without a story. Right now REQ-001 to REQ-010 all appear covered, but none of the NFRs link to a story.

5. Example rewrite

Before (REQ-005): "The system must enforce each restaurant's morning ordering deadline (for example, 10:30 for Kram Snack and 23:30 for Dar Lella as configured) and reject or hide orders placed after it."

After:

- **REQ-005a:** The system shall store an ordering cut-off time per restaurant, editable only by an administrator.

- **REQ-005b:** The system shall reject any order submitted after the cut-off of its restaurant and display "Ordering is closed for today".
- **REQ-005c:** The system shall show restaurants past their cut-off as "Closed" in the restaurant list instead of hiding them.
- Rationale: restaurants need a fixed cut-off to prepare exact quantities. Verification: automated test with a fixed clock one second before and one second after the cut-off.

Suggested next steps

1. Confirm the 23:30 question and the delivery-day rule with the stakeholders.
2. Add the missing access-control and menu-management requirements.
3. Split compound requirements and number the NFRs.
4. Add the edge-case acceptance criteria and a traceability table.

If you want, I can produce the full revised document with these changes applied.

Step B — Self-Critique Prompting

> Here is my full requirements draft (functional + non-functional + user stories): [pasted, same draft]. 1. List which ones are vague or ambiguous, 2. Rewrite them to be specific, measurable, and testable, 3. Then explain how your changes improve quality.

Output B:

Requirements review: vague items, rewrites, and why it helps

The non-functional requirements are mostly measurable already. The weak spots are in the functional requirements and the acceptance criteria, so most of this review covers those.

1. Vague or ambiguous items

Item	Problem
REQ-001	"Register and log in" has no limits: no code expiry (only NFR 6 says 5 min), no retry limit, no definition of a valid number.
REQ-003	"Daily availability" is undefined (who sets it, by what time?). No stated page behaviour.
REQ-004	"Noon delivery" has no tolerance window. "Pre-order for that day" is unclear if the cart is empty or the item sells out.
REQ-005	"Reject or hide" is two behaviours, so it cannot be tested. "Morning deadline" is not a defined time. Time zone is not stated. The example mixes 10:30 with 23:30, which is not "morning".
REQ-006 / REQ-007	"Tunisian e-payment" and "Tunisian payment gateway" are unnamed. "Successfully authorized" has no timeout or failure handling.
REQ-008	"(for example, ...)" leaves the status list open. "View" has no latency target.
REQ-009	"Dashboard" has no refresh rate, sort order or format. "Address or workplace" is ambiguous.

Item	Problem
REQ-010	"Update status" does not say which transitions are allowed or who can make them.
NFR 3	"Without errors" is absolute and hard to test. "Response-time increase" has no baseline.
NFR 8	"Only as needed" is subjective.
NFR 9	The usability test has no sample size or task definition.
NFR 10	"Usable in French and Arabic" does not say whether that includes right-to-left layout and full translation.
US-1 AC1-3	"Valid number", "clear error message" and "before it expires" are not quantified.
US-2 AC2	"Unavailable (or hidden)" is an either/or.
US-3 AC3	"Rejected (or the restaurant is hidden)" is an either/or.
US-4 AC3	"Told the payment failed" gives no wording, timing or retry limit.
US-5 AC2	"Can see the updated status when I open the order" does not say how fast it appears.
US-6 AC1	"Only my restaurant's" is good, but "confirmed orders for today" does not define the day boundary.
All stories	Story titles say "nearby", "follow", "run", which are not testable. No story covers admin or restaurant availability management.

2. Rewrites

Functional

- **REQ-001:** The system shall let an employee register and log in with a Tunisian mobile number (+216, 8 digits) verified by a 6-digit SMS code. The code shall be delivered within 30 seconds for 95% of requests and expire after 5 minutes. After 5 wrong attempts the number shall be locked for 15 minutes. A new code can be requested at most once per 60 seconds.
- **REQ-003:** The system shall display every active restaurant in the zone with its menu, prices in TND (3 decimals) and a per-item "available today" flag. Restaurant staff shall be able to set availability until the restaurant's deadline. Changes shall show to employees within 10 seconds.
- **REQ-004:** The system shall allow a cart with items from exactly one restaurant, from 1 to 20 items. Placing the order shall create a delivery for that day, with the delivery window of 12:00-13:00 (adjust to the real agreement). Orders with an empty cart or an unavailable item shall be refused with a specific error.
- **REQ-005:** Each restaurant shall have a configurable cut-off time (in Africa/Tunis time). From that moment the system shall refuse new orders for that restaurant (HTTP 409, message "Ordering closed for today") and mark the restaurant "closed" in the list. Kram Snack: 10:30; Dar Lella: 23:30 (confirm; this likely belongs to the previous day).
- **REQ-006/007:** At checkout the employee shall pick exactly one method: cash on delivery or [named gateway, e.g. Konnect/Paymee]. An e-payment order shall be confirmed only on a gateway "authorized" response. If no response arrives within 60 seconds, the order shall stay "pending" and be cancelled automatically after 15 minutes (this needs a stakeholder decision, as no cancelled state exists today).

- **REQ-008:** The system shall show a confirmation screen within 2 seconds of confirmation, with restaurant, items, total, payment method and address. Statuses are exactly: placed, preparing, out for delivery, delivered. The employee's order list shall show a change within 30 seconds. History shall keep orders for at least 12 months.
- **REQ-009:** The dashboard shall list today's confirmed orders (00:00-23:59 Africa/Tunis) for that restaurant only, grouped by workplace (company name plus address), showing per-item totals, order total, and "cash to collect" or "paid online" per order. It shall refresh at least every 30 seconds.
- **REQ-010:** Restaurant staff shall move an order only forward: placed → preparing → out for delivery → delivered. Staff shall mark a cash order "paid" only after it is "delivered". Each change shall record user and timestamp, and shall be refused with an error if it skips or reverses a step.

Non-functional

- **NFR 3:** At 500 concurrent users and 1,000 orders in 15 minutes, the order error rate shall stay under 0.1% and p95 response time shall stay within 150% of the baseline measured at 50 users.
- **NFR 8:** The system shall store only these fields: name, phone, workplace, delivery address. A restaurant sees only the customers with orders at that restaurant. Access shall be logged. Compliance shall be checked in a review against Law 2004-63 before launch.
- **NFR 9:** In a test with at least 10 first-time users, at least 9 shall place an order in under 3 minutes without help, and the SUS score shall be at least 75.
- **NFR 10:** All screens shall be fully translated into French and Arabic, with right-to-left layout for Arabic. The test list is: Chrome, Safari, Firefox and Edge (latest 2 versions), at 360 px width.

User-story acceptance criteria (examples)

- **US-1 AC3:** When the code is wrong or expired, the user stays logged out and sees "Incorrect or expired code" plus a "Send new code" button within 1 second.
- **US-2 AC2:** An unavailable item shall be shown greyed out with the label "Unavailable today", and its add-to-cart button shall be disabled.
- **US-3 AC3:** After the cut-off the restaurant shall appear as "Closed", and a direct order attempt returns "Ordering closed for today". One behaviour, not two.
- **US-4 AC3:** On a gateway decline, no order is confirmed, the message "Payment failed" appears within 3 seconds, and the user can retry up to 3 times or switch to cash.
- **US-5 AC2:** A status change by the restaurant appears in the employee's order view within 30 seconds, with no manual refresh.
- **US-6 AC1:** The dashboard shows only orders where restaurant_id matches the staff member's restaurant (a test with two restaurants shall confirm no leakage).

3. How the changes improve quality

1. **Testable.** Each rewrite has a number, a state or a named outcome, so a tester can write a pass/fail case. "Clear error message" becomes an exact message within 1 second.
2. **No either/or behaviour.** "Reject or hide" forced developers and testers to guess. One behaviour means one test and no disagreement at review.
3. **Fewer hidden assumptions.** Naming the time zone, delivery window, gateway and status list removes questions that would otherwise surface late in development.

4. **Traceable.** Concrete IDs, limits and statuses map cleanly from requirement to story to test case, which supports the project's traceability report.
5. **Safer for cost and risk.** Brute-force limits, timeouts and state rules close security and payment-consistency gaps early, when changes are cheapest.
6. **Flagged decisions.** Items marked "confirm" (the 23:30 cut-off, auto-cancel of unpaid orders, the delivery window) are open questions for stakeholders, not guesses I made on their behalf. Adding a cancelled state, for example, needs a stakeholder decision first.

Values such as 20 items, 15 minutes and 12 months are proposals to confirm, not facts from your draft.

Comparison

Criterion	A — "review and make them better"	B — self-critique (list → rewrite → explain)
What it produced	A narrative review: problems to fix, gaps, suggestions	A table of 19 vague items, a rewrite of each, then 6 reasons the rewrites improve quality
Clarity improvement	Spotted the most important **inconsistencies** (23:30 is not a "morning" deadline; "passwords" in an SMS-code system; status lists that disagree) and **gaps** (rate limits, restaurant accounts)	Rewrote items into **testable** statements: 6-digit code, 30 s delivery, 5 tries then 15 min lock, one behaviour instead of "reject or hide", time zone Africa/Tunis, updates visible within 30 s
Explained its reasoning?	Partly, per item, but no overall rationale	**Yes, explicitly:** testability, no either/or, fewer hidden assumptions, traceability, earlier risk reduction, decisions flagged
Honesty about invented values	Not stated	States that values like 20 items or 15 minutes are proposals to confirm, and flags stakeholder decisions (e.g. a cancelled state)

Verdict: B improved clarity more and explained its reasoning; A found more cross-requirement inconsistencies. The structured self-critique turns "make it better" into a checkable procedure (find → fix → justify). A's free review was better at spotting contradictions *between* requirements, so the two are complementary: we would run a free review for consistency, then a self-critique for testability.

Conclusion

1. **Structure in the prompt became structure in the answer.** Each technique gave a more usable SRS section than the plain prompt. The biggest gains came from **few-shot** (measurable NFRs) and **one-shot** (consistent, prioritised FRs). **Providing a concept** helped least, because the plain answer was already broad.

2. **Chaining keeps the AI inside the agreed scope.** The single-shot user stories invented features (like order cancellation) that our specification deliberately leaves open. Feeding the model our own requirements first prevented that.
3. **Self-critique turns review into a procedure.** It produced testable rewrites and explained why. A free review still found contradictions it missed, so we would use both.
4. **AI-invented numbers are assumptions, not facts.** Targets like 500 users, SUS ≥ 75 or a 15-minute lock look precise but came from the model. They must be confirmed with stakeholders before they enter the SRS.

Compared with our SRS (Lab 2). Our SRS already uses the patterns these techniques push towards: IDs with priority and rationale, NFRs with numbers (95 % under 500 ms), Given/When/Then-style acceptance criteria, and a review that rewrote vague verbs. The activity surfaced a few points worth adding later: rate-limiting SMS codes (our app already rate-limits sign-in), stating the time zone in the deadline requirement, and a defined delivery window. We record these as suggestions, not changes, since the SRS is a submitted deliverable.